

Project 3902 Math

Saurav Poudyel

March 2025

Character Motion: Velocity Calculation

A character's motion in a two-dimensional plane can be represented by a velocity vector $\mathbf{v}$. Given an orientation angle θ , the velocity is computed as follows:

$$\mathbf{v} = v \begin{bmatrix} \sin(\theta) \\ -\cos(\theta) \end{bmatrix}$$

Derivation:

This setup is common in game development to ensure intuitive direction handling:

- For $\theta = 0$: the character moves upward.
- For $\theta = \frac{\pi}{2}$: the character moves rightward.

This formula aligns with common screen coordinate systems where the y-axis is inverted relative to standard mathematical convention.

Example Calculation:

For a speed $v = 250$ and angle $\theta = \frac{\pi}{4}$ (45 degrees):

$$\mathbf{v} = 250 \begin{bmatrix} \sin\left(\frac{\pi}{4}\right) \\ -\cos\left(\frac{\pi}{4}\right) \end{bmatrix} = 250 \begin{bmatrix} 0.707 \\ -0.707 \end{bmatrix} = \begin{bmatrix} 176.8 \\ -176.8 \end{bmatrix}$$

Rotational Transformations: Bounding Box

The rotated bounding box is obtained by applying rotation matrices to the corner points of an object's local coordinate space.

Corners in local space (centered at the origin):

$$\mathbf{p}_1 = \begin{bmatrix} -\frac{w}{2} \\ -\frac{h}{2} \end{bmatrix}, \quad \mathbf{p}_2 = \begin{bmatrix} \frac{w}{2} \\ -\frac{h}{2} \end{bmatrix}, \quad \mathbf{p}_3 = \begin{bmatrix} \frac{w}{2} \\ \frac{h}{2} \end{bmatrix}, \quad \mathbf{p}_4 = \begin{bmatrix} -\frac{w}{2} \\ \frac{h}{2} \end{bmatrix}$$

Rotation by angle θ uses the matrix:

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Thus, the rotated position for a corner point $\mathbf{p}_i$ is:

$$\mathbf{p}_i^{rot} = R(\theta)\mathbf{p}_i + \mathbf{p}_{center}$$

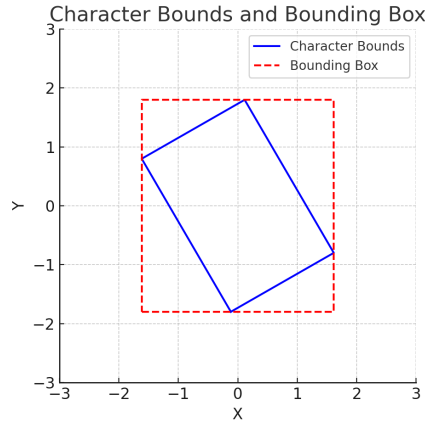


Figure 1: Rotated bounding box representation

Line of Sight for Mobs

In AI behavior, line-of-sight calculations help determine whether a mob can "see" the player. Raycasting is commonly used for this.

A common method follows the equation:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \quad t \geq 0$$

where:

- $\mathbf{o}$ is the mob's position.
- $\mathbf{d}$ is the direction to the target.

2D Raycasting Performance in MonoGame.

Plane Homing and Orbiting Behavior

The plane enemy follows a dynamic orbiting path while gradually homing toward the player's last known position. Its behavior is governed by three key mathematical concepts:

- **Linear Interpolation (LERP)**: Smoothly moves the orbit center toward the player's position.

- **Circular Motion Equations:** Keep the plane moving in a circular path around the orbit center.
- **Rotation Adjustment:** Aligns the plane's facing direction with its motion.

1. Updating the Orbit Center

To avoid instantly snapping the orbit center to the player, we use LERP:

$$C_{\text{new}} = (1 - t)C_{\text{old}} + tP$$

where:

- C_{old} is the current orbit center.
- P is the player's last known position.
- $t = \text{accelerationFactor} \times \Delta t$, with Δt being the frame time.

This produces a smooth, exponentially decaying approach toward P . Such techniques are widely used in game AI for smooth transitions

2. Orbit Angle Progression

The orbit angle is updated to simulate circular motion:

$$\theta_{\text{new}} = \theta_{\text{old}} + \Delta\theta$$

where:

$$\Delta\theta = \omega \cdot \Delta t \quad \text{and} \quad \omega = \frac{25\pi}{180} \cdot \text{orbitSpeedFactor}$$

This yields:

$$\theta_{\text{new}} = (\theta_{\text{old}} + \Delta\theta) \mod 2\pi$$

This ensures the angle wraps around after a full circle.

3. Position Update Using Circular Motion

Using the updated orbit center (x_c, y_c) and orbit angle θ , the new position is computed as:

$$x = x_c + r \cos(\theta)$$

$$y = y_c + r \sin(\theta)$$

where:

- r is the orbit radius.
- θ is the current orbit angle.

4. Adjusting Plane Orientation

To make the plane face its direction of motion (i.e., toward the orbit center), we calculate the desired rotation:

$$\theta_{\text{rotation}} = \tan^{-1} \left(\frac{y_c - y}{x_c - x} \right) + \frac{\pi}{2}$$

This rotation aligns the plane's front with the vector from its current position to the orbit center.

Final Summary of Equations

- **Orbit Center Update (LERP):**

$$C_{\text{new}} = (1 - t)C_{\text{old}} + tP, \quad t = \text{accelerationFactor} \times \Delta t$$

- **Orbit Angle Update:**

$$\theta_{\text{new}} = (\theta_{\text{old}} + \Delta\theta) \bmod 2\pi, \quad \Delta\theta = \frac{25\pi}{180} \cdot \text{orbitSpeedFactor} \cdot \Delta t$$

- **Circular Motion for Position:**

$$x = x_c + r \cos(\theta), \quad y = y_c + r \sin(\theta)$$

- **Rotation Adjustment:**

$$\theta_{\text{rotation}} = \tan^{-1} \left(\frac{y_c - y}{x_c - x} \right) + \frac{\pi}{2}$$

Implementing Orbital Motion in MonoGame.